THESIS · SLM (TWO-WAY ASL ↔ ENGLISH)

The SLM Code Architecture and Algorithm, Explained

What the program does from the moment it starts to the text on each screen: which function calls which, why, and what the data looks like at every step, with real values from our trained models.

1. **The big picture**: the four layers of the code
2. **The architecture figure**: which function calls which
3. **The algorithm**: the program written as numbered steps
4. **Hearing → Deaf, box by box**: "Where is your mom?" → *where mother*
5. **Deaf → hearing, box by box**: "home" → *mother is home*
6. **Active Context** across a whole conversation
7. **Inside the two models**: the neural network layers
8. **How the models were made**, glossary, and questions your prof may ask

34 signs · 91-word lookup table · 95 token IDs · word picker 75,411 parameters · TinyLlama 133,376 parameters · Vosk small English model (40 MB, offline)

1 · The big picture

The SLM is a translator that sits between two people. It has **two paths** that share one memory:

- **Hearing → Deaf (ND2D)**: the hearing person speaks; the Deaf person reads the **sign words as text** on their screen ("where mother").
- **Deaf → hearing (D2ND)**: the Deaf person signs; the hearing person reads an **English sentence as text** on their screen ("mother is home").

The code is organised in four layers. A layer only calls the layer below it; the foundation is used by everyone.

1 · Interface — what people touch

`full_demo.py`	the live demo: menu, the two screens
`speech_input.py`	microphone (sounddevice) + Vosk speech-to-text

▼ calls ▼

2 · Pipeline — the adviser's boxes

`slm_pipeline.py`	SLMPipeline: one method per box · non_deaf_to_deaf() · deaf_to_non_deaf() · ActiveContext · to_declarative() · covered()

▼ calls ▼

3 · Models — the learned parts

`word_picker.py + encoder_distilbert.py`	Hearing → Deaf model (75,411 numbers) + restrict_to_signs()
`decoder_tinyllama.py`	Deaf → hearing model (133,376 numbers)
`trained_picker.pt · trained_tinyllama.pt`	the saved learned numbers

▼ calls ▼

4 · Foundation — shared by everything

`vocab_pool.py`	the 34 signs, helper words, categories
`tokenizer.py`	word ↔ ID (95 IDs)
`onnx_config.py · make_labels.py`	synonyms (GLOSS_MAP), normalize_word()

One sentence per path

ND2D *shrinks* a full English sentence down to the words that have a sign, in sign order. **D2ND** *grows* a few signs into a full English sentence. **Active Context** connects the two by remembering who the conversation is about.

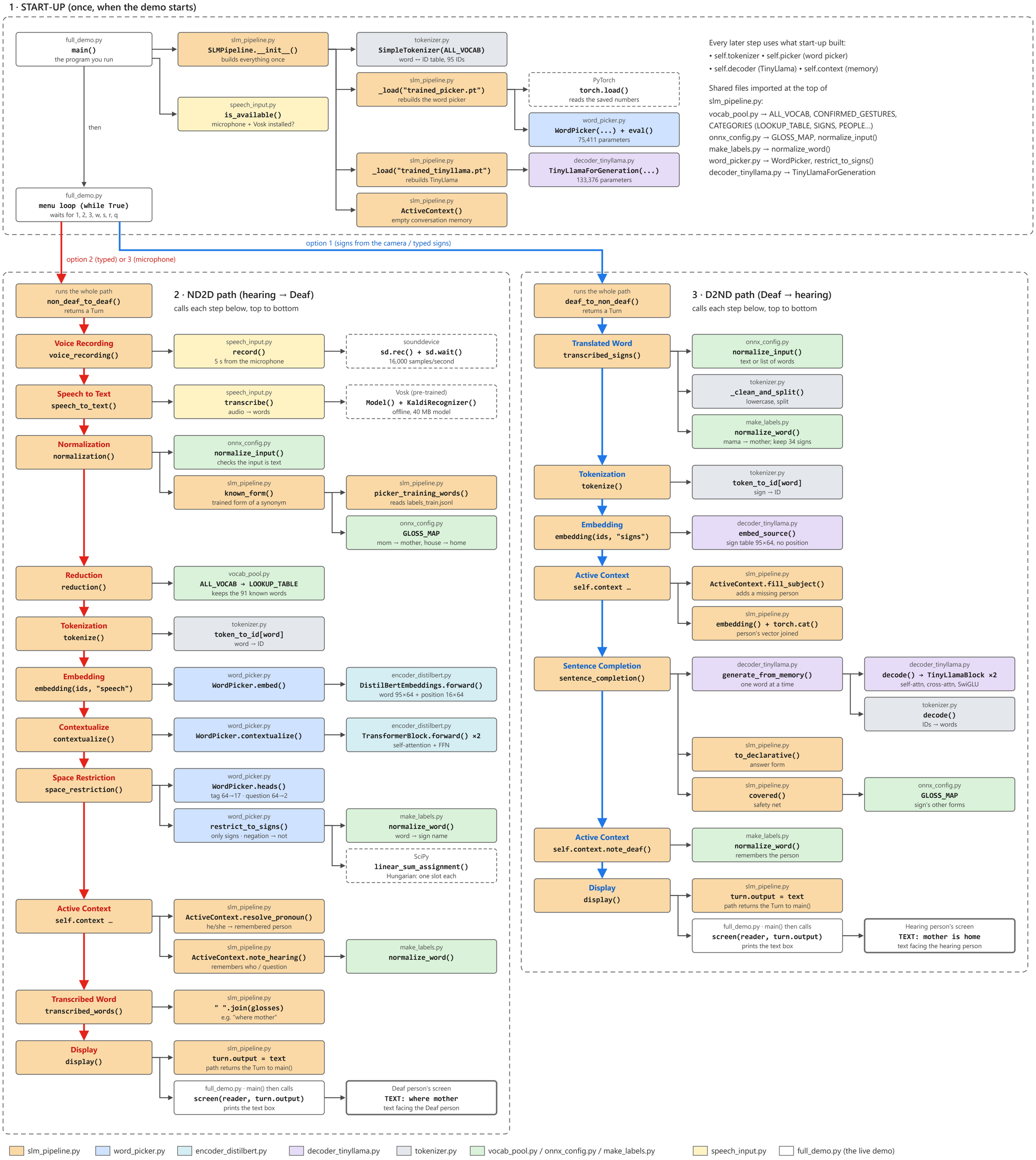
How the pipeline boxes map to the code

Every box in your adviser's pipeline is **one method** of the class `SLMPipeline` in `slm_pipeline.py`, named after the box. Two "runner" methods call the boxes in order: `non_deaf_to_deaf()` and `deaf_to_non_deaf()`. Each returns a `Turn`: the text for the screen plus what every box produced, which the demo prints step by step.

Pipeline box	Method in slm_pipeline.py	What it really calls
Voice Recording	`voice_recording()`	speech_input.record() → sounddevice
Speech to text	`speech_to_text()`	speech_input.transcribe() → Vosk
Normalization	`normalization()`	normalize_input(), known_form(), GLOSS_MAP
Reduction	`reduction()`	LOOKUP_TABLE from vocab_pool.py
Tokenize	`tokenize()`	tokenizer.token_to_id
Embedding	`embedding()`	picker.embed() or decoder.embed_source()
Contextualize	`contextualize()`	picker.contextualize() → TransformerBlock ×2
Space Restriction	`space_restriction()`	picker.heads(), restrict_to_signs(), Hungarian
Active Context	`self.context.…`	ActiveContext methods (Algorithm 6)
Transcribed Words	`transcribed_words()` / `transcribed_signs()`	join / clean the signs
Sentence Completion	`sentence_completion()`	decoder.generate_from_memory(), to_declarative(), covered()
Display	`display()`	turn.output → full_demo.screen()

2 · The architecture: which function calls which

Read top to bottom. Thick red/blue arrows = the next pipeline step. Thin grey arrows = "this function calls that function". Box colour = the file the function lives in. Dashed boxes = libraries we did not write.



How to read the architecture figure

- **Section 1, start-up**, runs once when `full_demo.py` starts: it builds the tokenizer, loads both trained models from their `.pt` files with `torch.load()`, creates the empty memory, and checks whether the microphone and Vosk are ready. Then the menu waits.
- **Section 2 (red)** and **Section 3 (blue)** run every time someone talks. The menu chooses one: option 2 or 3 → red path; option 1 → blue path.
- In each lane, the **first column** (orange) is the pipeline box and its method. The **second column** is what that method calls. The **third column** is what *that* function calls in turn.
- Colours tell you the file: orange = slm_pipeline.py, blue = word_picker.py, teal = encoder_distilbert.py, purple = decoder_tinyllama.py, grey = tokenizer.py, green = vocab_pool / onnx_config / make_labels, yellow = speech_input.py, white = full_demo.py.
- The thick-bordered boxes at the bottom are the **outputs: text on each person's screen**.

Section 1, start-up, step by step

What happens in the second between running `full_demo.py` and seeing the menu:

Call	What happens	Why
`SLMPipeline.__init__()`	Creates the one pipeline object.	Everything below is built once, so every later turn is fast.
`SimpleTokenizer(ALL_VOCAB)`	Builds token_to_id: <pad> = 0, <bos> = 1, <eos> = 2, <unk> = 3, then the 91 words in alphabetical order → 95 IDs.	Both models must use exactly the numbering they were trained with.
`_load("trained_picker.pt")` → `torch.load()`	Reads the saved file: the model's settings (64 numbers per word, 4 heads, 2 blocks, 16 slots) and its 75,411 learned numbers. An empty WordPicker is built with those settings and load_state_dict() pours the learned numbers in.	Training is slow and done once (offline); the demo only reloads the result.
`.eval()`	Switches off dropout (randomly silencing numbers, used only while training).	The same input must always give the same output.
`_load("trained_tinyllama.pt")`	The same for TinyLlama: 133,376 learned numbers.	
`ActiveContext()`	person = none, no question words, turns = 0.	A fresh conversation.
`speech_input.is_available()`	Checks that sounddevice and vosk can be imported and the Vosk model folder exists.	If not, option 3 is switched off and you type instead.
`menu loop`	Waits for a line: 1, 2, 3, w, s, r or q; the words may follow on the same line ("2 where is your mom").	

One turn, in one breath

After start-up, every turn follows the same shape: `main()` reads your line → calls one runner method → the runner calls its pipeline boxes top to bottom, each box calling helpers in other files → the runner returns a `Turn` → `main()` prints the steps and the screen box. The memory (`ActiveContext`) is the only thing that survives from one turn to the next.

3 · The algorithm

The same program written as numbered steps. ▷ starts a comment. ← means “becomes”.

Algorithm 1 · Start-up and the main loop — full_demo.py: main()

```
 1  slm ← SLMPipeline()                                ▷ START-UP, runs once
 2      tokenizer ← SimpleTokenizer(ALL_VOCAB)         ▷ 91 words + 4 special = 95 IDs
 3      picker    ← _load("trained_picker.pt")         ▷ word picker, 75,411 numbers
 4      decoder   ← _load("trained_tinyllama.pt")      ▷ TinyLlama, 133,376 numbers
 5      context   ← ActiveContext()                    ▷ empty conversation memory
 6  mic ← speech_input.is_available()                  ▷ are Vosk + microphone ready?
 7  repeat forever:
 8      read a line, e.g.  "2 where is your mom"
 9      if option = 1:  turn ← slm.deaf_to_non_deaf(signs)       reader ← HEARING person
10      if option = 2:  turn ← slm.non_deaf_to_deaf(text)        reader ← DEAF person
11      if option = 3:  turn ← slm.non_deaf_to_deaf()            ▷ records the microphone
12      if option = w / s / r / q:  list words / steps on-off / reset memory / quit
13      print every step of turn                       ▷ format_turn()
14      screen(reader, turn.output)                    ▷ the TEXT the reader sees
```

Algorithm 2 · Hearing → Deaf — slm_pipeline.py: SLMPipeline.non_deaf_to_deaf(text)

```
Input : typed text, or nothing (then the microphone is used)
Output: a Turn whose .output is the text for the Deaf person's screen
 1  if no text was typed:
 2      audio ← voice_recording()              ▷ 5 s × 16,000 samples/s   speech_input.record
 3      text  ← speech_to_text(audio)          ▷ pre-trained Vosk         speech_input.transcribe
 4  words ← normalization(text)                ▷ lowercase, no punctuation, don't → do not, mom → mother
 5  kept  ← reduction(words)                   ▷ only the 91 lookup-table words survive (max 16)
 6  if kept is empty:  show "" and stop
 7  ids   ← tokenize(kept)                     ▷ word → ID
 8  X     ← embedding(ids, "speech")           ▷ n × 64 numbers  (word meaning + position)
 9  H     ← contextualize(X)                   ▷ 2 transformer blocks: each word reads the whole sentence
10  g     ← space_restriction(H, kept)         ▷ Algorithm 3: keep/drop, order, "how", "not"
11  g     ← context.resolve_pronoun(words, g)  ▷ he/she → the remembered person
12  context.note_hearing(words, g)             ▷ remember the person and the question words
13  out   ← transcribed_words(g)               ▷ "where mother"
14  display(out)                               ▷ → Deaf person's screen
```

Algorithm 3 · Space Restriction — word_picker.py: restrict_to_signs(tag_scores, question_scores, words)

```
Input : S = n × 17 scores (column 0 = DROP, columns 1..16 = slot), q = 2 scores, the n words
 1  is_question ← (q[1] > q[0])                    ▷ should "how" be put in front?
 2  keep ← [ ];  negated ← false
 3  for each word w_i:
 4      if w_i is a negation (not, don't, no, never, can't …):  negated ← true;  skip it
 5      if normalize_word(w_i) is not one of the 34 signs:     skip it   ▷ the reader has no sign for it
 6      if max(S[i, 1..16]) > S[i, 0]:  add i to keep              ▷ the model says KEEP
 7  M ← S[keep, 1..k]                              ▷ k kept words × slots 1..k
 8  slots ← linear_sum_assignment(M)               ▷ Hungarian: every word its own slot, best total score
 9  out ← kept words sorted by slot
10  if negated:      out ← ["not"] + out
11  if is_question:  out ← ["how"] + out
12  return out
```

Algorithm 4 · Deaf → hearing

slm_pipeline.py: SLMPipeline.deaf_to_non_deaf(recognized)

```
Input : the sign recognizer's words, e.g. "home"
Output: a Turn whose .output is the English sentence for the hearing person's screen
 1  signs ← transcribed_signs(recognized)      ▷ clean, mama → mother, keep only the 34 signs
 2  if signs is empty:  show "" and stop
 3  ids   ← tokenize(signs)
 4  M     ← embedding(ids, "signs")             ▷ k × 64 (TinyLlama's sign table, no position)
 5  signs, added ← context.fill_subject(signs)  ▷ add the remembered person if a subject is missing
 6  if added:  M ← [embedding(added) ; M]       ▷ torch.cat: the person's 64 numbers go in front
 7  sentence ← sentence_completion(M, signs)    ▷ Algorithm 5
 8  context.note_deaf(signs, sentence)          ▷ remember the person
 9  display(sentence)                           ▷ → hearing person's screen
```

Algorithm 5 · Sentence Completion

slm_pipeline.py: sentence_completion() → decoder_tinyllama.py: generate_from_memory()

```
Input : M = the embedded signs (k × 64), the signs
 1  y ← [<bos>]                                  ▷ start-of-sentence marker
 2  repeat at most 16 times:
 3      scores ← decode(M, y)                    ▷ 2 TinyLlama blocks → a score for all 95 words
 4      next ← the word with the highest score   ▷ greedy choice
 5      if next = <eos>:  stop                   ▷ end-of-sentence marker
 6      y ← y + [next]
 7  sentence ← tokenizer.decode(y)               ▷ IDs → words
 8  if the hearing person's last turn was a question:
 9      sentence ← to_declarative(sentence)      ▷ "is mother home" → "mother is home"
10  if some sign is missing from the sentence:   ▷ covered() safety net
11      sentence ← the signs themselves          ▷ never show a fluent sentence about something else
12  return sentence
```

Algorithm 6 · Active Context (the memory both paths share)

slm_pipeline.py: class ActiveContext

```
memory: person, last_question_words, last_was_question, addressed_to_you, turns

resolve_pronoun(spoken, glosses)                          ▷ hearing → Deaf
 1  if a person is remembered, no person is in glosses, and "he"/"she" was said,
    and the remembered person fits the pronoun (he: father, teacher, friend;
    she: mother, teacher, friend):
 2      insert the person after any leading question word or "not"   ▷ "how" → "how mother"

note_hearing(spoken, glosses)
 3  last_question_words ← question words said (what, where, why, how)
 4  addressed_to_you ← true if "you" was said; false if a person was named instead
 5  person ← the last person mentioned (if any)

fill_subject(signs)                                       ▷ Deaf → hearing
 6  if a person was signed, or "my" was signed, or nobody is remembered:  change nothing
 7  needs a subject if: a question sign, OR a state sign without an activity,
    OR only place signs answering a "where" question
 8  if it needs one and the last question was not asked to "you":
 9      put the remembered person in front            ▷ "home" → "mother home"

note_deaf(signs, sentence)
10  clear the question memory;  person ← the last person in the signs/sentence
```

4 · Hearing → Deaf, box by box

Real run: the hearing person says **"Where is your mom?"**. Each card shows what the box receives, what it does, why it is needed, what it gives, and the real values from our trained word picker.

1 Voice Recording

`voice_recording() → speech_input.record() → sd.rec() + sd.wait()`

Receives	Nothing — it is used with menu option 3 (speak).
Does	Records 5 seconds from the microphone: 16,000 samples per second, one channel, 16-bit. That is 80,000 numbers describing the sound wave.
Why	The hearing person's voice has to be captured before anything can understand it.
Gives	Raw audio (bytes).

REAL EXAMPLE

With option 2 (typing) this step is skipped and the typed sentence is used as the text.

2 Speech to Text

`speech_to_text(audio) → speech_input.transcribe() → Vosk Model + KaldiRecognizer`

Receives	The raw audio.
Does	Feeds the audio to the pre-trained Vosk recognizer in chunks of 8,000 bytes (¼ second). Vosk returns the words it heard. The 40 MB model is loaded once and kept in memory.
Why	Our models read words, not sound. Vosk is offline, small enough for a Raspberry Pi, and was already trained on English speech, so we did not have to train speech recognition.
Gives	Text, e.g. "where is your mother".

REAL EXAMPLE

Windows' voice said "Where is your mother?" into a 16 kHz file; Vosk heard exactly *where is your mother*.

3 Normalization

`normalization(text) → normalize_input(), CONTRACTIONS, known_form() → GLOSS_MAP`

Receives	The text.
Does	Lowercases; removes punctuation; writes out short forms (don't → do not, i'm → i am); turns a possessive into its word (dad's → father); replaces a synonym the picker never saw in training with the word it knows (mom → mother, house → home, classroom → class). Words the picker did learn, like "okay", are left as they are.
Why	The same meaning must reach the model in the same form it was trained on. "mom" was never in the training sentences, so the model would not know it.
Gives	A list of clean words.

REAL EXAMPLE

"Where is your mom?" → **where is your mother** [mom -> mother]

4 Reduction

`reduction(words) → LOOKUP_TABLE (vocab_pool.ALL_VOCAB)`

Receives	The clean words.
Does	Keeps only the words that are in the lookup table — the 91 words the SLM knows (34 signs + 57 helper words like is, the, your) — and removes everything else. At most 16 words are kept, the model's limit.
Why	A word outside the lookup table would become <unk> (unknown) and confuse the model. Your adviser's design: reduce first, then tokenize.
Gives	The kept words (and the removed ones, for display).

REAL EXAMPLE

Nothing removed here. In “where is your mother, darling” the word *darling* would be removed.

5 Tokenize

`tokenize(kept) → tokenizer.token_to_id[word]`

Receives	The kept words.
Does	Looks up each word's ID in the dictionary built at start-up. IDs 0–3 are special (<pad>, <bos>, <eos>, <unk>); the 91 words follow in alphabetical order, so there are 95 IDs.
Why	A neural network computes with numbers, not letters.
Gives	A list of IDs.

REAL EXAMPLE

where → **88** · is → **41** · your → **94** · mother → **50**

6 Embedding

`embedding(ids, "speech") → WordPicker.embed() → DistilBertEmbeddings.forward()`

Receives	The IDs.
Does	Takes row *ID* of the word table (95 × 64) and row *position* of the position table (16 × 64), adds them, and normalizes (LayerNorm). Every word becomes a list of 64 numbers.
Why	An ID is only a label — 50 is not “bigger” than 41. The 64 learned numbers describe what the word means and where it stands; words used alike get similar numbers. Position matters here because spoken English word order carries meaning.
Gives	A table of 4 words × 64 numbers.

REAL EXAMPLE

word	ID	word table row (first 4 of 64)	+ position row	= embedding (after LayerNorm)
where	88	`1.09, -0.24, -0.41, 1.97, …`	`1.65, -0.76, -0.72, 0.64, …`	`2.15, -0.78, -1.09, 1.96, …`
is	41	`0.06, 2.35, -0.43, -0.36, …`	`-0.20, 1.24, 0.61, 1.84, …`	`-0.26, 2.58, -0.05, 1.00, …`
your	94	`-1.00, 1.36, -0.46, -1.31, …`	`0.54, -0.28, 0.60, 0.72, …`	`-0.73, 0.82, -0.17, -0.93, …`
mother	50	`-0.02, 0.17, 1.19, 2.28, …`	`0.55, -1.60, -0.45, 0.35, …`	`0.39, -0.85, 0.46, 1.87, …`

7 Contextualize

`contextualize(X) → WordPicker.contextualize() → TransformerBlock.forward() ×2`

Receives	The 4 × 64 embedding.
Does	Two transformer blocks. In each, every word compares itself with every other word (self-attention, 4 "heads" looking for different things) and mixes in information from the words it finds relevant; then a small feed-forward network refines each word. The input is added back after each part (residual), so nothing is lost.
Why	Whether a word should be kept depends on the words around it: "your" in "your mother" only points at mother, but "you" in "how are you" is the person asked about.
Gives	A new 4 × 64 table: the same words, now aware of the whole sentence.

REAL EXAMPLE

Attention in block 2 (average of the 4 heads): how much each word looks at each other word

word ↓ looks at →	where	is	your	mother
where	31%	30%	29%	11%
is	27%	12%	17%	44%
your	8%	8%	13%	71%
mother	52%	10%	15%	23%

"your" looks 71% at "mother": the model has learned that "your" belongs to the person after it. "mother" looks 52% at "where": it knows it is the thing being asked about.

word	before (embedding)	after Contextualize
where	`2.15, -0.78, -1.09, 1.96, …`	`3.00, 2.06, 2.76, 6.20, …`
is	`-0.26, 2.58, -0.05, 1.00, …`	`4.48, 2.03, -2.33, -3.20, …`
your	`-0.73, 0.82, -0.17, -0.93, …`	`4.79, 1.13, -2.72, -6.40, …`
mother	`0.39, -0.85, 0.46, 1.87, …`	`2.35, -5.45, 1.10, 5.36, …`

8 Space Restriction

`space_restriction(H, kept) → WordPicker.heads() + restrict_to_signs() → normalize_word(), linear_sum_assignment()`

Receives	The contextualized words.
Does	The **tag head** gives every word 17 scores: DROP, or keep at slot 1–16. The **question head** decides whether "how" must be added. Then the rules of Algorithm 3: a negation becomes "not", only words that have a sign can survive, and the **Hungarian algorithm** gives every kept word its own slot.
Why	The Deaf reader should only see words that have a sign, in sign order. The model alone could put two words in the same slot; the Hungarian algorithm makes that impossible.
Gives	The glosses (sign words in order).

REAL EXAMPLE

word	P(drop)	best slot	has a sign?	decision
where	0%	slot 1: 100%	yes	**slot 1**
is	100%	—	no	**DROP**
your	100%	—	yes (your → you)	**DROP**
mother	0%	slot 2: 100%	yes	**slot 2**

P(add "how") = 0%. Kept: where (slot 1), mother (slot 2) → **where mother**. Note "your": it maps to the sign "you", so the rules would allow it — it was dropped only because the model, after reading the sentence, scored DROP at 100%.

9 Active Context

`context.resolve_pronoun() + context.note_hearing()`

Receives	The spoken words and the glosses.
Does	Replaces he/she with the remembered person when it fits; then remembers the last person mentioned, the question words, and whether the question was asked to "you".
Why	A conversation continues across turns: "Is she okay?" only makes sense if the system remembers who "she" is.
Gives	The final glosses; the memory is updated.

REAL EXAMPLE

Turn 1 remembers **mother**. Later, "Is she okay?" → the picker gives only *how* ("she" has no sign) → Active Context adds the remembered person → **how mother**.

10–11 Transcribed Words → Display

`transcribed_words() → " ".join · display() → turn.output → full_demo screen()`

Does	Joins the glosses with spaces and stores them as turn.output. When the path returns, full_demo.py prints them in a box titled DEAF PERSON'S SCREEN.
Gives	TEXT facing the Deaf person.

REAL EXAMPLE

```
DEAF PERSON'S SCREEN   where mother
```

5 · Deaf → hearing, box by box

Real run, right after turn 1: the Deaf person answers by signing only **“home”**.

1 Transcribed / Translated Word

`transcribed_signs(recognized) → normalize_input(), _clean_and_split(), normalize_word()`

Receives	The sign recognizer's words — for now typed in the demo (option 1).
Does	Accepts a string or a list, lowercases and splits it, maps label variants to sign names (mama → mother), keeps only the 34 signs, sets everything else aside; at most 16 signs.
Why	TinyLlama was trained only on the 34 sign names.
Gives	The signs.

REAL EXAMPLE

“home” → **home**

2 Tokenize

`tokenize(signs) → token_to_id`

Does	Same dictionary as the other path: sign → ID.

REAL EXAMPLE

home → **36**

3 Embedding

`embedding(ids, "signs") → decoder.embed_source()`

Does	Takes row *ID* of TinyLlama's own sign table (95 × 64). **No position is added.**
Why no position	The recognizer may deliver signs in a different order (“mother home” or “home mother”); the meaning comes from WHICH signs were made, so TinyLlama reads them as a set.
Gives	k signs × 64 numbers — called the MEMORY.

4 Active Context

`context.fill_subject() → embedding() + torch.cat()`

Does	If the signs need a subject and none was signed, adds the remembered person, and puts that person's 64 numbers in front of the memory.
Why	A Deaf person answering “where is your mother?” often signs only “home”.

REAL EXAMPLE

Memory had **mother**, the last question used “where”, and the only sign is a place → added 'mother' from earlier. Memory is now 2 × 64:

sign	its 64 numbers (first 4)
mother	`-0.35, 0.36, 0.51, 1.09, …`
home	`-2.41, -1.27, 1.19, -0.35, …`

5 Sentence Completion

`sentence_completion() → generate_from_memory() → decode() → TinyLlamaBlock ×2; tokenizer.decode(); to_declarative(); covered()`

Does	TinyLlama writes the sentence one word at a time (Algorithm 5). Each time, it reads the words written so far (self-attention) and the signs (cross-attention), scores all 95 words, and takes the best. Then two rules: the answer form, and the safety net.
Why	Signs have no grammar words (is, the, at); the sentence has to be completed into English a hearing person reads naturally.

REAL EXAMPLE

#	written so far	chosen	top 3 choices	cross-attention: looks at
1	(start)	**is**	is 100% · this 0% · i 0%	mother 66% · home 34%
2	is	**mother**	mother 100% · teacher 0% · father 0%	mother 100% · home 0%
3	is mother	**home**	home 100% · at 0% · fine 0%	mother 21% · home 79%
4	is mother home	**<eos>**	<eos> 100% · tomorrow 0% · now 0%	mother 11% · home 89%

Raw sentence: **is mother home**. The hearing person's last turn was a question, so `to_declarative()` makes it an answer → **mother is home**. `covered()` checks every sign is in it: mother ✓ home ✓.

6 Active Context → Display

`context.note_deaf() · display() → full_demo screen()`

Does	Remembers the person for the next turn, stores the sentence as turn.output, and full_demo.py prints it in a box titled HEARING PERSON'S SCREEN.
Gives	TEXT facing the hearing person.

REAL EXAMPLE

```
HEARING PERSON'S SCREEN   mother is home
```

6 · Active Context across a whole conversation

One real conversation of 8 turns, run through the pipeline without resetting the memory. Watch the memory column: the remembered person is carried from turn to turn.

Turn	Screen shows	Active Context did	Memory after the turn
Hearing person says "Where is your mom?"	**where mother**	remembers mother	person = **mother**; question words = where; asked to "you" = no
Deaf person signs "home"	**mother is home**	added 'mother' from earlier	person = **mother**; question words = —; asked to "you" = no
Hearing person says "Is she okay?"	**how mother**	he/she -> mother; remembers mother	person = **mother**; question words = how; asked to "you" = no
Deaf person signs "fine"	**mother is fine**	added 'mother' from earlier	person = **mother**; question words = —; asked to "you" = no
Hearing person says "I do not understand, can you say that again?"	**not understand again**	remembers mother	person = **mother**; question words = —; asked to "you" = yes
Deaf person signs "please again"	**please say it again**	this turn does not need a subject	person = **mother**; question words = —; asked to "you" = yes
Hearing person says "What time is class tomorrow?"	**what time class tomorrow**	remembers mother	person = **mother**; question words = what; asked to "you" = yes
Deaf person signs "school tomorrow"	**i will go to school tomorrow**	this turn does not need a subject	person = **mother**; question words = —; asked to "you" = yes

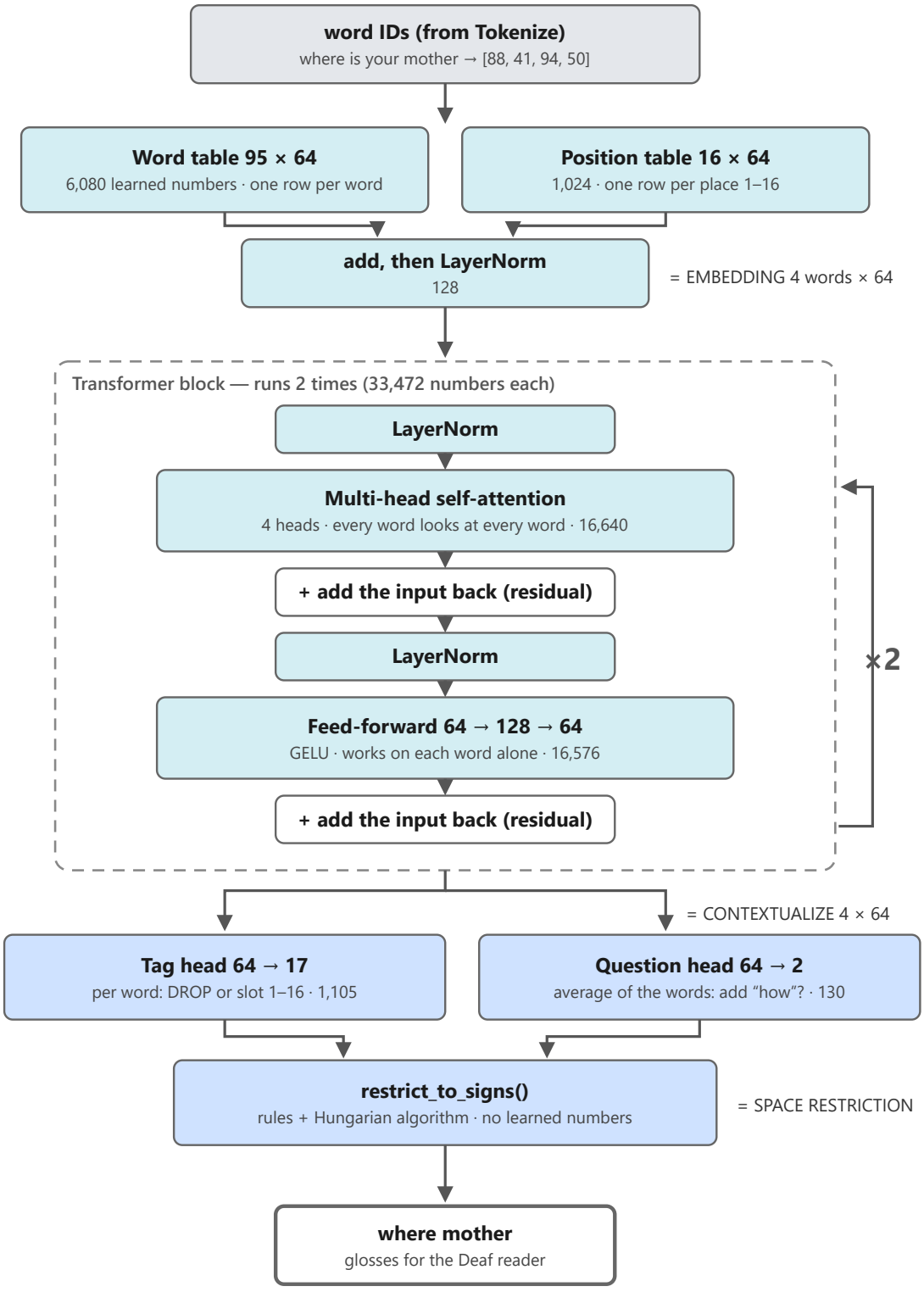
- **Turn 2**: only "home" was signed; the last question was "where" about mother, so mother is added → *mother is home* (and turned into an answer, not a question).
- **Turn 3**: "she" has no sign, so the picker outputs only "how"; Active Context resolves she → mother.
- **Turn 5**: "not" is shown so the meaning is not inverted; "you" was said, so the memory notes the question was asked to the Deaf person, and the following turns do not add mother.

7 · Inside the two models

Both models are small transformers with the same size settings: 64 numbers per word, 4 attention heads, 2 blocks, a feed-forward layer of 128. They are different *shapes* because their jobs are opposite.

The word picker (hearing → Deaf): an encoder, DistilBERT-style

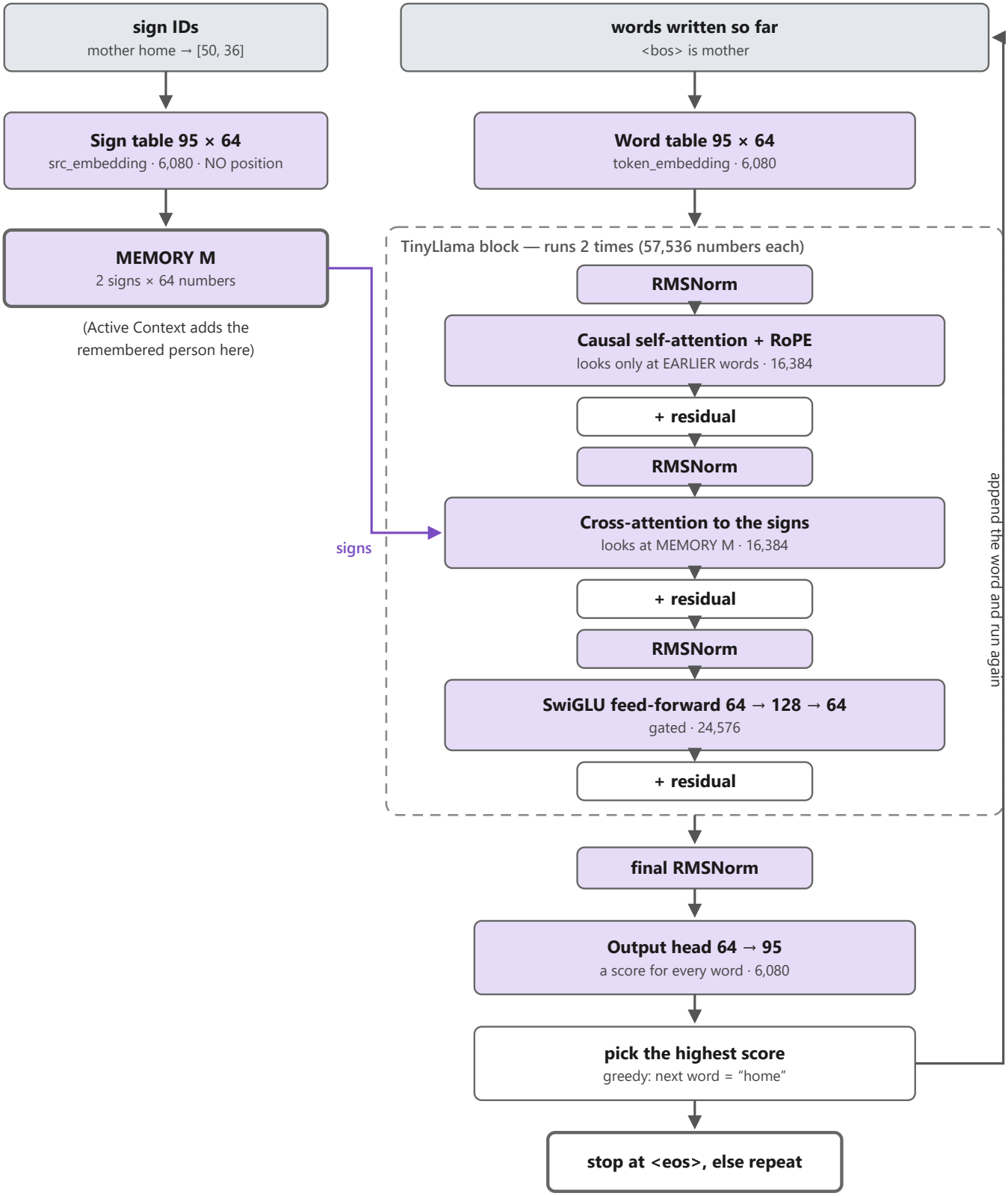
It reads the **whole sentence at once** (every word can see every other word) and makes one decision per word. It covers three pipeline boxes: Embedding, Contextualize and Space Restriction.



Total learned numbers (parameters): 75,411

TinyLlama (Deaf → hearing): a decoder, Llama-style

It **writes** the sentence one word at a time. Left: the signs become the memory. Right: the words written so far go through the blocks; cross-attention lets them read the signs; the output head scores all 95 words and the best one is appended. It covers Embedding (signs) and Sentence Completion.



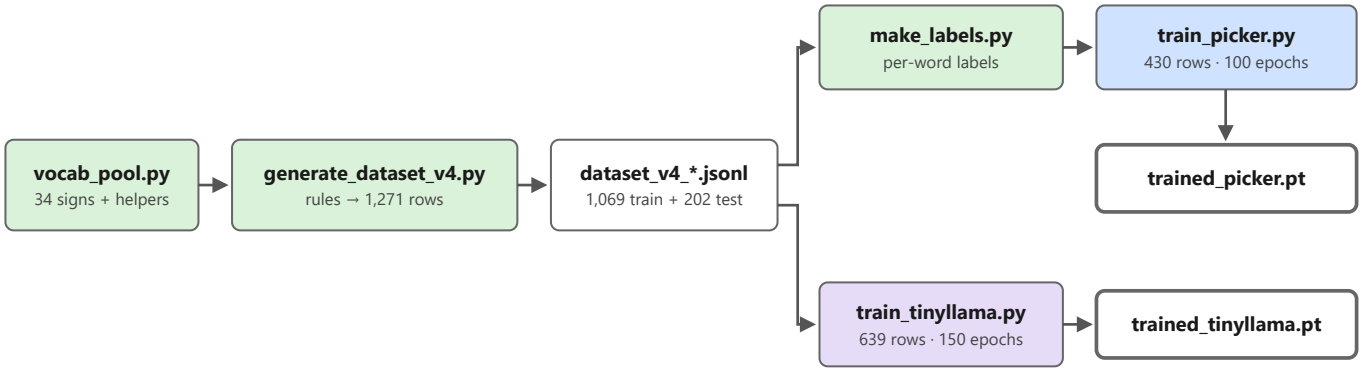
Total learned numbers (parameters): 133,376

Where the learned numbers are

Part	Word picker	TinyLlama
Embedding tables	word 6,080 + position 1,024 + LayerNorm 128	sign 6,080 + word 6,080 (position by RoPE, no table)
One block	33,472 (attention 16,640 · feed-forward 16,576 · norms 256)	57,536 (self-attn 16,384 · cross-attn 16,384 · SwiGLU 24,576 · norms 192)
Blocks	2 → 66,944	2 → 115,072
Output	tag head 1,105 + question head 130	final norm 64 + output head 6,080
Total	**75,411**	**133,376**

8 · How the models were made (offline, before the demo)

These files do not run during a conversation; they produced the two `.pt` files the demo loads.



- **generate_dataset_v4.py** builds the training sentences from **rules over categories** (PERSON + STATE → "mother is fine", ACTIVITY + TIME → "i will study tomorrow"…), so adding a sign to a category covers it everywhere. It refuses to save data with contradictions, unknown words, or non-sign outputs.
- **make_labels.py** turns each Reduce example into a per-word label (0 = drop, 1–16 = slot) and checks that the answer can be rebuilt from the labels (0 failures).
- **train_picker.py** (100 epochs) and **train_tinyllama.py** (150 epochs) adjust the learned numbers until the loss is near zero. Both use a fixed random seed, so training is repeatable.
- 202 rows are **held out**, never trained on, to measure accuracy honestly: Deaf → hearing 24/24 (100%), hearing → Deaf 172/178 (96.6%).

Glossary

Token / ID	A word's number in the dictionary (tokenizer.py). "mother" = 50.
Vector, embedding	A list of 64 learned numbers that stands for a word. Words used alike end up with similar lists.
n × 64	A table: n rows (one per word), 64 numbers each. This is the shape data has inside the models.
Parameter (learned number)	One number inside a model that training adjusted. The picker has 75,411; TinyLlama 133,376.
Transformer block	The repeating unit of both models: attention (words look at each other) + feed-forward (each word refined) + residual connections.
Self-attention	Each word scores how relevant every other word is and mixes in their information. "4 heads" = 4 separate ways of looking.
Causal attention	Attention that may only look at EARLIER words — TinyLlama cannot peek at the word it is about to write.
Cross-attention	TinyLlama's words looking at the signs (the memory). This is how the signs steer the sentence.
Residual connection	Adding a block's input back to its output, so information is refined rather than replaced.

LayerNorm / RMSNorm	Rescale each word's 64 numbers to a steady size so training stays stable. RMSNorm is the simpler Llama version.
GELU / SwiGLU	The non-linear functions inside the feed-forward parts. SwiGLU adds a “gate” that decides how much passes through.
RoPE	Rotary position encoding: TinyLlama tells word positions apart by rotating the numbers by an angle per position.
Logits → probability	Raw scores; softmax turns them into percentages that add up to 100%.
Greedy decoding	Always take the highest-scoring next word.
Hungarian algorithm	A classic method (SciPy linear_sum_assignment) that matches words to slots one-to-one with the best total score.
Epoch / loss	One pass over all training rows / how wrong the model still is (lower is better).
Held-out test	Sentences kept out of training, used only to measure accuracy honestly.

Questions your prof may ask

Why are there two different models?

The two directions are opposite jobs. Hearing → Deaf SHRINKS a sentence into signs: an encoder that sees the whole sentence at once is ideal for deciding per word keep or drop. Deaf → hearing GROWS signs into a sentence: a decoder that writes one word at a time is ideal for producing grammar words that were never signed.

Can the hearing → Deaf model invent words?

No. It can only pick from the words that were said. The only two words it may add are “how” (the question flag) and “not” (a spoken negation).

Why does Reduction come before Tokenize?

Your adviser's design. Words outside the 91-word lookup table are removed first, so the model only ever sees words it was trained on.

Why the Hungarian algorithm?

The model scores each word separately, so two words could both prefer slot 1. The Hungarian algorithm gives every kept word a different slot while keeping the total score as high as possible.

Why does TinyLlama ignore sign order?

The recognizer may output signs in any order; “mother home” and “home mother” mean the same. Reading the signs as a set makes the sentence the same either way.

How does the system remember who is being talked about?

ActiveContext stores the last person mentioned by either side. It replaces he/she on the hearing side and adds a missing subject on the Deaf side.

What if TinyLlama writes a sentence that leaves out a sign?

covered() checks every sign; if one is missing the screen shows the signs themselves rather than a fluent but wrong sentence.

How accurate is it?

On held-out sentences the models never saw: Deaf → hearing 24/24 (100%), hearing → Deaf 172/178 (96.6%). Conversation test: all 34 signs exercised, 45/50 match, 3 new, 2 small differences.

What can it not do yet?

There is no sign for who, when, his/her or no: those words are dropped, and a negation is shown as the word “not”. It handles the sentence patterns it was trained on; very different sentences may be reduced imperfectly.

Explaining the architecture in one minute

> “When the program starts, `main()` in full_demo.py creates the SLM pipeline: it builds the word-to-ID dictionary, loads our two trained models and creates an empty memory. When the hearing person speaks, Vosk turns the voice into text; Normalization and Reduction clean it and keep only the words we know; Tokenize and Embedding turn them into numbers; our word picker reads the whole sentence, decides which words have a sign and in what order, and the Hungarian algorithm fixes the order. Active Context replaces he or she with the remembered person, and the Deaf person reads the result as text. When the Deaf person signs, the signs are turned into numbers, Active Context adds a missing person from memory, and TinyLlama writes the English sentence one word at a time, which the hearing person reads as text.”